

9. PreparedStatement

The PreparedStatement interface is a subinterface of Statement. It is used to execute parameterized query. which gives you added functionality with a couple of advantages over a generic Statement object. This statement gives you the flexibility of supplying arguments dynamically.

Let's see the example of parameterized query:

```
String sql = "insert into emp values(?,?,?)";
```

As you can see, we are passing parameter (?) for the values. Its value will be set by calling the setter methods of PreparedStatement.

Why use PreparedStatement?

Improves performance: The performance of the application will be faster if you use PreparedStatement interface because query is compiled only once.

How to get the instance of PreparedStatement?

The prepareStatement() method of Connection interface is used to return the object of PreparedStatement.

Syntax:

```
public PreparedStatement prepareStatement (String query) throws SQLException{ }
```

Creating PreparedStatement Object

All parameters in JDBC are represented by the ? symbol, which is known as the parameter marker. You must supply values for every parameter before executing the SQL statement.

The setXXX() methods bind values to the parameters, where XXX represents the Java data type of the value you wish to bind to the input parameter. If you forget to supply the values, you will receive an SQLException.

Each parameter marker is referred by its ordinal position. The first marker represents position 1, the next position 2, and so forth. This method differs from that of Java array indices, which starts at 0.

All of the Statement object's methods for interacting with the database (a) execute(), (b) executeQuery(), and (c) executeUpdate() also work with the PreparedStatement object. However, the methods are modified to use SQL statements that can input the parameters.

Closing PreparedStatement Object

Just as you close a Statement object, for the same reason you should also close the PreparedStatement object.

A simple call to the `close()` method will do the job. If you close the `Connection` object first, it will close the `PreparedStatement` object as well. However, you should always explicitly close the `PreparedStatement` object to ensure proper cleanup.

Methods of PreparedStatement interface

The important methods of `PreparedStatement` interface are given below:

Method	Description
<code>public void setInt(int paramIndex, int value)</code>	sets the integer value to the given parameter index.
<code>public void setString(int paramIndex, String value)</code>	sets the String value to the given parameter index.
<code>public void setFloat(int paramIndex, float value)</code>	sets the float value to the given parameter index.
<code>public void setDouble(int paramIndex, double value)</code>	sets the double value to the given parameter index.
<code>public int executeUpdate()</code>	executes the query. It is used for create, drop, insert, update, delete etc.
<code>public ResultSet executeQuery()</code>	executes the select query. It returns an instance of <code>ResultSet</code> .

Example of PreparedStatement interface that inserts the record

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class PreparedStatementInsertExample {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
```

```
        "jdbc:mysql://localhost:3306/organization", "root", "root");

        PreparedStatement stmt = con.prepareStatement(
            "INSERT INTO employee values(?, ?, ?, ?)");

        stmt.setInt(1, 6);
        stmt.setString(2, "Rocky");
        stmt.setString(3, "Team Lead");
        stmt.setInt(4, 90000);

        int count = stmt.executeUpdate();

        System.out.println(count + " records inserted");

        con.close();
    } catch (Exception e) {
        System.out.println(e);
    }
}
```

Output:

```
1 records inserted
```

Example of PreparedStatement interface that updates the record

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class PreparedStatementUpdateExample {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/organization", "root", "root");

            PreparedStatement stmt = con.prepareStatement(
                "UPDATE employee SET name=? WHERE id=?");

            stmt.setString(1, "Arul");
            stmt.setInt(2, 6);

            int count = stmt.executeUpdate();

            System.out.println(count + " records updated");

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output:

```
1 records updated
```

Example of PreparedStatement interface that deletes the record

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class PreparedStatementDeleteExample {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/organization", "root", "root");

            PreparedStatement stmt = con.prepareStatement("DELETE FROM employee
                                                         WHERE id=?");

            stmt.setInt(1, 6);

            int count = stmt.executeUpdate();

            System.out.println(count + " records deleted");

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output:

```
1 records deleted
```